Software Requirements Specification
Project: Jomnouy

Prepared for: Institute of Technology of Cambodia
Prepared by: [Your Name]
Date: October 20, 2026

Table of Contents

1. Introduction
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Intended Audience and Reading Suggestions
    - 1.4 Definitions, Acronyms, and Abbreviations
2. Overall Description
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Classes and Characteristics
    - 2.4 Operating Environment
    - 2.5 Design and Implementation Constraints
3. Specific Requirements
    - 3.1 External Interface Requirements
    - 3.2 System Features & User Stories
    - 3.3 Non-Functional Requirements
    - 3.4 Database Requirements
4. Supporting Information & Appendices
    - 4.1 Analysis Models (ERD & Architecture)
    - 4.2 API Contract
    - 4.3 UI Wireframes

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to detail the technical, functional, and non-functional requirements for "Jomnouy" (meaning "helper" in Khmer). This document serves as the authoritative blueprint for the development, deployment, and testing of the application over the course of the 14-week individual project timeline. It outlines the architecture, database schema, and specific user stories required to achieve a passing grade.

1.2 Scope

Jomnouy is a full-stack, community-driven web application designed to facilitate mutual aid, local borrowing, and the exchange of second-hand goods. The system operates as a digital bulletin board utilizing five strict categories: Question, Help, Giveaway, Rent, and Sell. To maintain a focused scope and adhere to the project timeline, the system strictly prohibits native in-app monetary transactions. Instead, users coordinate rentals or sales manually via a built-in direct messaging inbox. The scope encompasses a Vue 3 Single Page Application (SPA), a NestJS backend REST API, and a PostgreSQL database managed by Prisma.

1.3 Intended Audience and Reading Suggestions

This document is intended for the course instructor evaluating the project milestones, the defense panel assessing the final architecture, and the sole developer building the system. Readers interested in the high-level goals should review Section 2 (Overall Description). Readers concerned with grading metrics and technical APIs should focus on Section 3 (Specific Requirements) and Section 4 (Appendices).

1.4 Definitions, Acronyms, and Abbreviations

- **CRUD:** Create, Read, Update, Delete. The four basic operations of persistent storage.
- **JWT:** JSON Web Token. The standard used for secure, stateless user authentication.
- **Listing:** A user-generated bulletin post categorized into one of five predefined types.
- **Simple Inbox:** A direct messaging system utilizing standard HTTP REST requests rather than real-time WebSockets.
- **Stretch Feature:** An advanced technical requirement (Search, Filter, Pagination) that accounts for a specific portion of the final grade.

1.5 References

- Chun, T. (2026). *Individual Project Guideline*. Institute of Technology of Cambodia (ITC).
- Vue.js Core Team. (2026). *Vue.js 3 Documentation*. https://vuejs.org/
- NestJS. (2026). *NestJS Official Documentation*. https://docs.nestjs.com/
- PostgreSQL Global Development Group. (2026). *PostgreSQL Documentation*. https://www.postgresql.org/docs/
- Microsoft. (2026). *TypeScript Handbook*. https://www.typescriptlang.org/docs/
- Jelvix. (n.d.). *Software Requirements Specification: Structure and AI Fit*.

2. Overall Description

2.1 Product Perspective

Jomnouy is a newly developed, standalone web application implementing a modern three-tier architecture. The presentation tier is a Vue 3 SPA utilizing Pinia for state management. This frontend consumes a NestJS application tier via REST API endpoints. Data persistence is handled by a PostgreSQL data tier. The system does not rely on third-party commercial APIs (e.g., Stripe, SendGrid), ensuring it remains entirely self-contained and free to host for the developer.

[PASTE USE CASE DIAGRAM HERE]
(Use the Mermaid Use Case code provided to generate the image)

2.2 Product Functions

The core functionalities of the Jomnouy platform include:

- **Account Management:** Secure registration and JWT login.
- **Categorized Listings:** Users can post under five distinct tags: Question, Help, Giveaway, Rent, Sell.
- **Anonymous Posting:** Users can optionally hide their username when posting sensitive questions or requests.
- **Direct Messaging:** A private, threaded inbox for users to coordinate physical meetups or share payment details safely off-feed.
- **Public Comments:** Open threads on listings for community discussion.

2.3 User Classes and Characteristics

User Class	Characteristics & Privileges
Standard User	Authenticated community members. They can execute CRUD operations exclusively on their own Listings, Comments, and Messages based on ownership validation matching their JWT.
Moderator	Administrative accounts configured in the database seed. They possess all User privileges plus the ability to delete any listing or comment to prevent commercial spam.
Anonymous Guest	Unauthenticated traffic. They can read the main listing feed but are blocked by API route guards from viewing private details or executing POST requests.

2.4 Operating Environment

The client application operates in modern web browsers (Chrome, Safari, Firefox). The backend and database will be containerized via Docker and executed via `docker-compose.yml`, ultimately deployed behind an Nginx reverse proxy on the institution's server.

2.5 Design and Implementation Constraints

The project is bound by the specific technologies mandated in the course rubric (Vue 3, NestJS, PostgreSQL). Furthermore, to ensure the project is delivered feature-complete by Week 14, a critical architectural constraint is placed on the messaging system: **The private direct messaging feature will utilize standard HTTP REST API endpoints (a "Simple Inbox" model) rather than real-time WebSocket connections.** The frontend will render threaded conversations via standard data fetching. E-commerce constraints dictate that all pricing (Rent/Sell) must be manually typed into the text description of a listing rather than processed via complex numerical columns and payment gateways.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The UI will utilize a clean, card-based layout optimized for mobile and desktop reading, similar to Reddit. Listings categorized as "Help" will feature a distinct visual badge if marked "Urgent." Forms will utilize client-side validation to prevent blank submissions. The inbox UI will display messages in a threaded format, with the user's messages aligned to the right and received messages aligned to the left.

3.1.2 Software Interfaces

The backend NestJS API interfaces with PostgreSQL strictly via Prisma Client queries. The frontend interfaces with the backend exclusively via asynchronous `fetch` or `axios` calls, expecting standard JSON objects representing DTOs (Data Transfer Objects).

3.2 System Features & User Stories

3.2.1 Core CRUD & Listing Management

Story: As a User, I want to create a listing by selecting a category (Question, Help, Giveaway, Rent, Sell) and writing a description so that I can interact with the community.
Acceptance Criteria: The backend validates the POST request. The `category` must match the required enum. The listing is saved and visible on the feed. The user can later toggle `is_resolved` to true, which visually closes the listing.

3.2.2 Anonymous Posting

Story: As a User, I want the option to post anonymously so that I can ask sensitive questions without revealing my identity.
Acceptance Criteria: The UI form includes an "Anonymous" boolean checkbox. If true in the database, the API response masks the user data, and the frontend renders the author as "Anonymous Neighbor."

3.2.3 Direct Messaging (Simple Inbox)

Story: As a User, I want to send a private message to a listing owner to coordinate meeting times or exchange bank details.
Acceptance Criteria: The user clicks "Message" on a post. A POST request saves a `DirectMessage` record containing `sender_id` and `receiver_id`. The Inbox UI performs a GET request to retrieve all messages associated with the user's ID, rendering them as a threaded conversation.

3.2.4 Search, Filter, and Pagination (Stretch Feature)

Story: As a User, I want to filter the main feed by category (e.g., Rent) and urgency so that I can easily find what I need without scrolling endlessly.
Acceptance Criteria: The `GET /listings` endpoint accepts query parameters (`?category=Rent&urgency=Urgent&page=1`). Prisma dynamically constructs the query utilizing `take` and `skip` for pagination. The API returns a maximum of 10 items alongside total count metadata.

3.3 Non-Functional Requirements

- **Security:** All user passwords are hashed via bcrypt or argon2. Unauthenticated endpoints reject modifications with a `401 Unauthorized`. Users attempting to edit data they do not own receive a `403 Forbidden`. No SQL injection vulnerabilities are present due to Prisma parameterized queries.
- **Validation:** Global validation pipes in NestJS ensure incoming JSON payloads conform strictly to defined DTOs. Invalid requests return a 400 Bad Request with an array of specific error messages.
- **Portability:** Containerization via Docker ensures the system runs identically on the developer's local machine and the ITC production server.

3.4 Database Requirements

The database must be relational (PostgreSQL) and structured around four core entities: User, Listing, Comment, and DirectMessage. Strict one-to-many relationships must be enforced via foreign keys. All database schema changes must be tracked via Prisma migration files.

4. Supporting Information & Appendices

4.1 Analysis Models

[PASTE ERD DIAGRAM HERE]
(Use the Mermaid ERD code provided to generate the database schema image)

4.2 API Contract (Core Endpoints)

Method	Endpoint & Query Params	Auth Required	Description	Expected Status Codes
POST	/api/auth/register	None	Registers a new user, hashes password, returns JWT	201 Created, 400 Bad Request
POST	/api/auth/login	None	Validates credentials, returns session JWT	200 OK, 401 Unauthorized
GET	/api/listings?category=[val]&page=[num]	None	Fetches paginated listings (Stretch Feature)	200 OK
POST	/api/listings	User	Creates a new listing under one of the 5 categories	201 Created, 401 Unauthorized
PATCH	/api/listings/:id	Owner / Mod	Updates listing text or toggles is_resolved	200 OK, 403 Forbidden
DELETE	/api/listings/:id	Owner / Mod	Hard deletes listing and cascades to comments	204 No Content, 403 Forbidden
POST	/api/messages	User	Creates a DirectMessage to another user	201 Created, 401 Unauthorized
GET	/api/messages/inbox	User	Retrieves message threads for the logged-in user	200 OK, 401 Unauthorized

4.3 UI Wireframe References

The following visual wireframes outline the intended user interface for the Vue 3 application.

[PASTE MAIN FEED WIREFRAME HERE]
(Showing category filter buttons and "Urgent" badges)

[PASTE DIRECT MESSAGE / INBOX WIREFRAME HERE]
(Showing left/right aligned chat bubbles)